



Unidad 2: Bases de Python y Venv

Materia: Seminario de lenguajes Python

Carrera: Licenciatura en Sistemas

Docentes: Mg. Lic. María Alejandra Vranic

Lic. Gonzalo Cerbelli

Año cursada: 2026

Índice

1. Python	2
2. El código	2
A. Declaración de variables	2
B. Sentencias condicionales (if / elif / else)	2
C. Estructura match (similar al switch de otros lenguajes)	3
D. Bucles for	3
E. Bucles while	3
F. Funciones	4
3. Manejo de excepciones	4
4. Venv	5

1. Python

Python es un lenguaje de programación de alto nivel, interpretado y de propósito general, creado por Guido van Rossum y lanzado en 1991. Su principal característica es la simplicidad en la sintaxis, lo que facilita la lectura y escritura del código. Gracias a esta claridad, Python se ha convertido en una de las opciones preferidas tanto por programadores principiantes como por desarrolladores profesionales.

Sus principales características son:

- **Sintaxis simple y legible:** se parece al lenguaje natural y permite escribir código de forma concisa.
- **Multiparadigma:** admite programación orientada a objetos, imperativa y funcional.
- **Interpretado:** el código se ejecuta línea por línea, sin necesidad de compilación previa.
- **Tipado dinámico:** no es necesario declarar el tipo de las variables explícitamente.
- **Gran biblioteca estándar:** incluye módulos para trabajar con archivos, redes, bases de datos, expresiones regulares, entre otros.
- **Portabilidad:** se puede ejecutar en múltiples sistemas operativos sin cambios en el código.
- **Comunidad amplia y activa:** ofrece numerosos recursos, librerías y soporte.

2. El código

A. Declaración de variables

En Python, no es necesario declarar el tipo de variable. El tipo se infiere automáticamente según el valor asignado. Por ejemplo:

```
nombre = "Juan"      # Cadena de texto (str)
edad = 25             # Número entero (int)
altura = 1.70         # Número decimal (float)
es_estudiante = True # Valor booleano (bool)
```

B. Sentencias condicionales (if / elif / else)

Permiten ejecutar bloques de código según si una condición es verdadera o falsa.

```
edad = 18

if edad >= 18:
    print("Eres mayor de edad.")
elif edad >= 13:
    print("Eres adolescente.")
else:
    print("Eres menor de edad.")
```

C. Estructura match (similar al switch de otros lenguajes)

Disponible a partir de Python 3.10. Permite comparar una variable con distintos casos posibles.

Ejemplo:

```
opcion = 2

match opcion:
    case 1:
        print("Elegiste uno.")
    case 2:
        print("Elegiste dos.")
    case _:
        print("Opción no válida.")
```

D. Bucles for

Se usa para iterar sobre secuencias como listas, cadenas o rangos.

```
for i in range(5): # range(5) genera los números 0, 1, 2, 3, 4
    print("Iteración:", i)
```

```
elementos = ["elemento1", "elemento2", "elemento3", "elemento4"]

for elemento in elementos:
    print("Elemento: ", elemento)
```

E. Bucles while

Se ejecutan mientras una condición sea verdadera.

```
contador = 0

while contador < 3:
    print("Contador: ", contador)
    contador += 1
```

F. Funciones

Las funciones permiten agrupar código reutilizable. Se definen con la palabra clave `def`.

```
def saludar(nombre):  
    print("Hola ", nombre)  
  
saludar("Carlos")
```

3. Manejo de excepciones

Una excepción en Python es un evento que ocurre durante la ejecución de un programa y que interrumpe su flujo normal cuando se presenta un error o una situación inesperada. En lugar de finalizar el programa abruptamente, Python permite capturar y manejar esas situaciones mediante un mecanismo especial llamado manejo de excepciones. Una excepción representa un error que puede ser previsto y tratado, lo que hace que el programa sea más seguro y robusto.

Python maneja las excepciones mediante una estructura de control llamada **try-except**. Esta permite detectar y manejar errores que ocurren durante la ejecución de un bloque de código, evitando que el programa se detenga abruptamente.

El bloque **try** contiene el código que puede generar una excepción. Si ocurre un error en ese bloque, Python busca un bloque **except** correspondiente para manejar esa excepción. Además, se pueden usar los bloques **else** y **finally** para controlar comportamientos adicionales:

- **else** se ejecuta solo si no se produce ninguna excepción en el `try`. Es útil cuando querés ejecutar algo solo si todo salió bien.
- **finally** se ejecuta siempre, haya ocurrido una excepción o no. Se usa típicamente para liberar recursos, cerrar archivos, o mostrar mensajes de cierre.

Un ejemplo de código podría ser:

```
try:  
    numero = int(input("Ingresa un número: "))  
    resultado = 10 / numero  
    print("Resultado:", resultado)  
except ZeroDivisionError:  
    print("No se puede dividir entre cero.")  
except ValueError:  
    print("Debes ingresar un número válido.")  
else:  
    print("La operación se realizó con éxito.")  
finally:  
    print("Fin del bloque de manejo de excepciones.")
```

En este ejemplo, se capturan dos tipos de errores comunes: dividir entre cero y convertir una entrada inválida a número entero. Así, Python permite manejar los errores de forma controlada, mejorando la robustez y estabilidad del programa.

4. Venv

Venv es la abreviación de Virtual Environment o Entorno Virtual. Es una herramienta incorporada en python que permite crear un entorno aislado para un proyecto. Dentro de este entorno, se pueden instalar librerías y paquetes sin que afecten al resto del sistema o a otros proyectos. Esto es especialmente útil cuando trabajás con múltiples aplicaciones que requieren versiones distintas de las mismas dependencias, o simplemente querés mantener tu entorno limpio y controlado.

El entorno virtual funciona como una carpeta separada que contiene una copia del intérprete de Python, junto con su propio directorio de paquetes. Cuando lo activás, todos los comandos que ejecutás (como instalar librerías con pip) afectan solo a ese entorno, y no al Python global del sistema.

Para crear un entorno virtual, desde la terminal o consola, debemos ubicarnos en la carpeta del proyecto y ejecutar el siguiente comando:

```
python -m venv .venv
```

Ese comando creará una carpeta llamada venv dentro del proyecto, que contendrá el entorno virtual. El nombre venv es una convención.

Para activar el entorno virtual, también desde la terminal y sobre el mismo directorio, debemos ejecutar el siguiente comando:

- En Windows:

```
.\.venv\Scripts\activate
```

- En Linux o macOS:

```
source .venv/bin/activate
```

Una vez activado, se podrá ver el nombre del entorno al principio de tu línea de comandos. A partir de ese momento, cualquier instalación con pip se hará dentro del entorno virtual.

Para desactivar el entorno, simplemente ejecutamos en la terminal el siguiente comando:

```
deactivate
```

Deberían verlo de la siguiente manera en la termina:

```
● PS C:\Users\Gonzalo\Documents\Repositorios\slp-api> python -m venv venv
● PS C:\Users\Gonzalo\Documents\Repositorios\slp-api> .\venv\Scripts\activate
● (venv) PS C:\Users\Gonzalo\Documents\Repositorios\slp-api> deactivate
○ PS C:\Users\Gonzalo\Documents\Repositorios\slp-api> 
```